



PROGRAMME:INFORMATION AND COMMUNICATION TECHNOLOGY (ICT)

DEPARTMENT:..... INFORMATION SYSTEMS

MODULE NAME: MOBILE APPLICATION DEVELOPMENT

COURSE CODE:.....IT 8203

FACILITATOR: RUKOIJU A

UQF 8 SECOND YEAR

Authors:

1. 24101133370111: ANDREA ALOYCE LAHOYA
2. 24101133370029: ISAACK IBRAHIM JUMA
3. 24101133370134: KASSIAN MASEBIUS NJOWOKA
4. 24101133370002: RAJABU JOSEPH MSEMAKWELI

CAMPUS ITEM LOCATOR

A Web-Based Lost and Found Management App for Campus

Table of Contents

1. Introduction	5
1.1 Purpose	5
2. Requirements	6
2.1 Functional Requirements	6
2.2 Non-functional Requirements	7
3. Design and Implementation	8
3.1 Overview	8
3.2 Use Case Diagram	9
3.3 Sequence Diagram	10
3.4 Database Schema	10
3.5 API Description	12
3.6 User Authentication	13
3.6.1 JWT Overview	13
3.6.2 Login Process	13
3.6.3 Security Measures	13

.....	14
3.7 Data Visualization	14
.....	14
4. Results	16
.....	16
4.1 Pages	16
.....	16
4.1.1 User Log-in Page	16
.....	16
4.1.2 User Dashboard	16
.....	16
4.1.3 View Items Page	17
.....	17
4.1.4 Upload Item Page	18
.....	18
4.1.5 Admin Pages	19
.....	19
5. Summary and Conclusion	21
.....	21
5.1 Pros and Cons of the Application	21
.....	21
5.2 Future Work	21
.....	21
5.3 Challenges During Development	21
.....	21
5.4 Other Potential Uses	21
.....	21
6. References	22
.....	22

Table of Figures

Figure 1: App Architecture Diagram
Figure 2: Use Case Diagram
Figure 3: Sequence Diagram
Figure 4: Database Schema Diagram
Figure 5: Login Page
Figure 6: Student Dashboard
Figure 7: Admin Dashboard

1 Introduction

1.1 Purpose

Campus Item Locator is a web-based application developed to manage lost and found items within a campus environment.

The system allows students to report lost or found items, upload images, and track the status of items. Administrators can monitor activities, manage students, and ensure proper system usage.

The main goal is to reduce loss of personal property and improve recovery efficiency.

2 Requirements

2.1 Functional Requirements

The system shall:

- Allow students to register accounts
- Allow students and admins to login
- Allow students to upload lost/found items
- Allow image upload for each item
- Allow students to mark items as found
- Allow admin to view all users
- Allow admin to block/unblock students
- Allow admin to view all items
- Store data securely in PostgreSQL

2.2 Non-functional Requirements

The system should:

- Be secure (JWT authentication & password hashing)
- Be responsive (mobile & desktop compatible)
- Provide fast response time
- Ensure data consistency
- Be scalable using cloud PostgreSQL (Neon)
- Be maintainable and structured

3 Design and Implementation

3.1 Overview

The system follows a client-server architecture:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js with Express
- Database: PostgreSQL (Neon Cloud)
- Authentication: JSON Web Token (JWT)

System Structure:

```
public/  
├── assets/icon.png  
├── uploads/  
├── base.html  
├── register.html  
├── student/  
└── admin/
```

```
server.js  
package.json  
.env
```

3.2 Use Case Diagram

Actors:

- Student
- Admin

Student Use Cases:

- Register
- Login
- Upload Item
- View Items
- Mark as Found

Admin Use Cases:

- Login
- View Students
- Block/Unblock
- View All Items

3.3 Sequence Diagram

Example: Upload Item

1. Student logs in
2. JWT token generated
3. Student submits item form
4. Image stored in /uploads
5. Item saved in PostgreSQL
6. Server returns success response

3.4 Database Schema

Users Table

- id
- name

- email
- password (hashed)
- role
- status
- created_at

Items Table

- id
- title
- description
- image
- status
- user_id (Foreign Key)
- created_at

Relationship:

One user can upload many items.

3.5 API Description

Authentication APIs

- POST /api/register
- POST /api/login

Item APIs

- POST /api/items
- GET /api/items
- GET /api/items/my
- PUT /api/items/:id

Admin APIs

- GET /api/admin/users
- PUT /api/admin/users/:id

3.6 User Authentication

3.6.1 JWT Overview

JWT is used for authentication.

After login:

- Token is generated
- Sent to client
- Stored in localStorage
- Used in Authorization header

3.6.2 Login Process

1. User enters credentials
2. Server verifies password
3. If correct → JWT generated
4. Redirect based on role

3.6.3 Security Measures

- Password hashing (bcrypt)
- JWT secret in .env
- Role-based access control
- Protected routes
- File validation for upload

3.7 Data Visualization

The dashboard displays:

- Total items uploaded
- Items marked as found
- Simple statistics overview

Future improvements include charts and analytics.

4 Results

4.1 Pages

4.1.1 User Log-in Page

- Email
- Password
- Login button
- Register link

4.1.2 User Dashboard

- Total uploaded items
- Items found
- Quick navigation

4.1.3 View Items Page

- List of all items
- Image display
- Status indicator

4.1.4 Upload Item Page

- Title
- Description
- Image upload
- Submit button

4.1.5 Admin Pages

Admin can:

- View students
- Block/unblock students
- View all items
- Monitor system statistics

5 Summary and Conclusion

5.1 Pros and Cons

Pros:

- Secure authentication
- Easy to use
- Organized structure
- Cloud database support

Cons:

- No real-time notifications
- No mobile app
- Limited analytics

5.2 Future Work

- Email notifications
- SMS alerts
- Advanced dashboard charts
- Search and filter improvements
- Mobile app development

5.3 Challenges During Development

- Implementing JWT authentication
- Managing image uploads
- Database relationship handling
- Role-based access implementation

5.4 Other Potential Uses

- School lost & found system
- Office asset tracking
- Hostel management system

- Library tracking system

6 References

- Node.js Documentation
- Express.js Documentation
- PostgreSQL Documentation
- JWT Documentation
- Multer Documentation